First Order Logic

```
def is_variable(x):
    return isinstance(x, str) and x[0].islower()

def parse_term(s):
    s = s.strip()

    if '(' not in s:
        return s

    name = s[:s.index('(')].strip()
    inside = s[s.index('(') + 1:-1]

    args = []
    depth = 0
    start = 0

    for i, ch in enumerate(inside):
        if ch == '(':
            depth += 1
        elif ch == ')':
            depth -= 1
        elif ch == ',' and depth == 0:
            args.append(parse_term(inside[start:i]))
            start = i + 1

    args.append(parse_term(inside[start:]))
    return (name, args)

def apply_substitution(x, theta):
    if is_variable(x) and x in theta:
        return apply_substitution(theta[x], theta)

    if isinstance(x, tuple):
        return (x[0], [apply_substitution(a, theta)
                       for a in x[1]])

    return x

def occurs(var, x, theta):
    x = apply_substitution(x, theta)
```

```
    if var == x:
        return True

    if isinstance(x, tuple):
        return any(occurs(var, a, theta) for a in x[1])

    return False

def unify(x, y, theta):
    x = apply_substitution(x, theta)
    y = apply_substitution(y, theta)

    if x == y:
        return theta

    if is_variable(x):
        if occurs(x, y, theta):
            return None
        theta[x] = y
        return theta

    if is_variable(y):
        if occurs(y, x, theta):
            return None
        theta[y] = x
        return theta

    if isinstance(x, tuple) and isinstance(y, tuple):
        if x[0] != y[0] or len(x[1]) != len(y[1]):
            return None

        for a, b in zip(x[1], y[1]):
            theta = unify(a, b, theta)

            if theta is None:
                return None

        return theta

    return None

def main():
    e1 = input("Enter first expression: ")
    e2 = input("Enter second expression: ")
```

```
    x = parse_term(e1)
    y = parse_term(e2)

    theta = unify(x, y, {})

    if theta is None:
        print("Unification failed")
    else:
        print("Unification successful")
        print("Substitution:", theta)def is_variable(x):
    return isinstance(x, str) and x[0].islower()

def parse_term(s):
    s = s.strip()

    if '(' not in s:
        return s

    name = s[:s.index('(')].strip()
    inside = s[s.index('(') + 1:-1]

    args = []
    depth = 0
    start = 0

    for i, ch in enumerate(inside):
        if ch == '(':
            depth += 1
        elif ch == ')':
            depth -= 1
        elif ch == ',' and depth == 0:
            args.append(parse_term(inside[start:i]))
            start = i + 1

    args.append(parse_term(inside[start:]))
    return (name, args)

def apply_substitution(x, theta):
    if is_variable(x) and x in theta:
        return apply_substitution(theta[x], theta)

    if isinstance(x, tuple):
        return (x[0], [apply_substitution(a, theta)
```

```
                    for a in x[1]])

    return x

def occurs(var, x, theta):
    x = apply_substitution(x, theta)

    if var == x:
        return True

    if isinstance(x, tuple):
        return any(occurs(var, a, theta) for a in x[1])

    return False

def unify(x, y, theta):
    x = apply_substitution(x, theta)
    y = apply_substitution(y, theta)

    if x == y:
        return theta

    if is_variable(x):
        if occurs(x, y, theta):
            return None
        theta[x] = y
        return theta

    if is_variable(y):
        if occurs(y, x, theta):
            return None
        theta[y] = x
        return theta

    if isinstance(x, tuple) and isinstance(y, tuple):
        if x[0] != y[0] or len(x[1]) != len(y[1]):
            return None

        for a, b in zip(x[1], y[1]):
            theta = unify(a, b, theta)

            if theta is None:
                return None
```

```
        return theta

    return None

def main():
    e1 = input("Enter first expression: ")
    e2 = input("Enter second expression: ")

    x = parse_term(e1)
    y = parse_term(e2)

    theta = unify(x, y, {})

    if theta is None:
        print("Unification failed")
    else:
        print("Unification successful")
        print("Substitution:", theta)
        print("Unified Expression:",
              apply_substitution(x, theta))

main()
        print("Unified Expression:",
              apply_substitution(x, theta))

main()
```

